НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
Міністерство освіти і науки України
Інститут комп'ютерних систем
Кафедра інформаційних систем

Протокол лабораторної роботи №1

«Розгортання середовища та порівняльний аналіз мовних моделей»

з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-243
Іпатов Богдан Сергійович
Варіант №1 (11 mod 10)

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Мета завдання: дослідити поведінку та накладні витрати інференсу двох типів локальних мовних моделей: інструктивної (llama3.2:3b) та міркувальної (qwen3:4b). Виконано прямі HTTP-запити до локального REST API Ollama на ендпойнт /api/generate. Спочатку виконано просте фактичне питання («Столиця України? Відповідай одним словом.»), після чого проведено серію з 5 запусків задачі на міркування (когнітивна задача про ручку і зошит).

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b (міркувальна)	llama3.2:3b (інструктивна)
response	Киев	Київ.
eval_count	2759	5
Час, с	25.05	0.05
Токенів/с	110.44	160.11

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b (міркувальна)	llama3.2:3b (інструктивна)
Правильних із 5	5 / 5	2 / 5 (помилка 10 грн)
eval_count (середнє)	1353.2	215.0
Час, с (середнє)	12.03	1.57

Типова помилка інструктивної моделі llama3.2:3b: у більшості запусків модель видає інтуїтивну відповідь «10 грн». Це відбувається через евристичне віднімання 110 - 100 = 10 без урахування того, що різниця у 100 грн розподіляється між двома доданками ($x + (x + 100) = 110 \Rightarrow 2x = 10 \Rightarrow x = 5$ грн).

Відповіді на контрольні питання (Крок 6):

1. Куди пішли додаткові токени в міркувальної моделі?

Відповідь: Додаткові токени (2759 токенів у qwen3:4b проти 5 токенів у llama3.2:3b) були витрачені на генерацію внутрішнього блоку роздумів (`thinking`). Модель детально розбирала географічний статус Києва, аналізувала альтернативні історичні столиці (Харків, Львів) і проводила багаторазову самоперевірку, перш ніж згенерувати фінальне слово.

2. На яких типах задач міркувальні моделі дають перевагу, а на яких їх використання невиправдане?

Відповідь: Міркувальні моделі забезпечують вирішальну перевагу в математичних задачах, написанні коду, логічних головоломках і плануванні дій агента, де потрібен покроковий аналіз (Chain of Thought). Натомість на простих фактичних

питаннях або прямій класифікації їх використання є абсолютно невиправданим через колосальну перевитрату часу і ресурсів (у 500 разів більше часу на одне слово).

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі навчаються методом Supervised Fine-Tuning (SFT) одразу видавати найбільш імовірну пряму відповідь. Міркувальні моделі проходять спеціальний етап навчання з підкріпленням (наприклад, алгоритмами сімейства GRPO/PPO), де модель винагороджується за генерацію розгорнутих ланцюжків міркувань, самокорекцію та верифікацію проміжних результатів.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, сама по собі кількість токенів свідчить лише про архітектурну схильність моделі до внутрішнього монологу. Перевага у розумінні проявляється лише тоді, коли ці токени дійсно містять валідні логічні виведення, але на простих питаннях багатослівність стає дефектом надлишковості.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між qwen3:4b та llama3.2:3b?

Відповідь: llama3.2:3b демонструє вищу швидкість генерації (~160 токенів/с проти ~110 токенів/с у qwen3:4b), оскільки має меншу кількість активних параметрів (3 млрд проти 4 млрд) та оптимізовану архітектуру шарів уваги, що вимагає менше обчислень FLOPS на один токен на GPU RTX 3070.

> ***Висновок за завданням 1:*** *Експеримент емпірично довів фундаментальну різницю між двома показниками: швидкістю генерації (токенів/с) та загальним часом відповіді (с). Швидкість генерації визначається залізом та розміром моделі, тоді як загальний час залежить від глибини роздумів, яку модель собі призначила. На простому питанні режим міркування спричинив 500-кратну затримку (25.05 с проти 0.05 с) без будь-якої користі у якості. Навпаки, у задачі на логіку саме механізм CoT дозволив qwen3:4b досягти 100% точності (5 з 5), тоді як інструктивна llama3.2:3b у більшості випадків піддалася когнітивній пастці. Практичний висновок для архітектури агентів: міркувальні моделі непридатні для простих проміжних кроків через високий latency, але критично необхідні для генерації складних планів.*

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Мета завдання: дослідити, як інструкція верхнього рівня (system message) впливає на поведінку моделі, довжину, мову та дотримання формату без зміни програмного коду чи перенавчання вагів. Використано варіант №1: «Поясни різницю між списком і кортежем у Python» для двох моделей та двох system-повідомлень: 1) «Ти лаконічний технічний асистент.», 2) «Відповідай лише українською, максимум двома реченнями.».

Таблиця 3 – Вплив system-повідомлення на вихідний текст

Модель	System	Довжина, симв.	Мова відповіді	К-ть речень	Виконано?
llama3.2:3b	лаконічний асистент	1323	укр. (суржик, англізми, нідерландська)	17	Ні (1323 симв.)
llama3.2:3b	українською, 2 речення	246	укр. (полонізм «różниця»)	2	Частково (2 реч.)
qwen3:4b	лаконічний асистент	131	літературна українська	2	Так (ідеально)
qwen3:4b	українською, 2 речення	184	літературна українська	2	Так (ідеально)

Таблиця 4 – Витрати токенів при зміні system-повідомлення

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	54	437	491
llama3.2:3b	українською, 2 речення	58	76	134
qwen3:4b	лаконічний асистент	55	1903	1958
qwen3:4b	українською, 2 речення	64	824	888

Відповіді на запитання розробника (Крок 6):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Модель qwen3:4b бездоганно вклалася в ліміт в обох випадках (рівно 2 змістовні речення). Модель llama3.2:3b на першому промпті згенерувала 17 речень (1323 символи), проігнорувавши лаконічність, і лише при прямій числовій вказівці змогла скоротити текст до 2 речень.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Так, у відповідях llama3.2:3b спостерігався важкий суржик, неіснуючі граматичні конструкції («обрбота», «сквіток»), а також прямі іншомовні вкраплення: нідерландське дієслово «kunnen» («Lists kunnen змінюватися») та польська літера «ó» («różниця»). Модель qwen3:4b писала бездоганною українською мовою.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: Значення prompt_tokens зросло (з 54 до 58 у llama та з 55 до 64 у qwen), оскільки друге повідомлення є довшим за кількістю слів та містить специфічні кириличні символи, які BPE-токенізатор розбиває на більшу кількість токенів.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: qwen3:4b генерує невидимий масив токенів роздумів (`thinking`). Навіть коли видима користувачеві відповідь складає всього 131 символ (~30 слів), модель витрачає 1903 completion_tokens на внутрішній аналіз відмінностей мутабельності та хешування.

Відповіді на питання чат-бота (Крок 7):

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови?

Відповідь: Це визначається балансом навчальних даних та наявністю етапу Reasoning-RL. Міркувальна модель у внутрішньому діалозі аналізує обмеження промпту перед виведенням, що підвищує дисципліну виконання інструкцій. Для малих інструктивних моделей жорсткі обмеження часто губляться в механізмі уваги.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі?

Відповідь: System message поміщається в спеціальний префіксний блок шаблону діалогу (chat template) зі спеціальними токенами `<|im_start|>system`. Модель натренована надавати цьому блоку статус глобального контексту, проте в процесі декодування токени user-повідомлення знаходяться ближче до позиції генерації і можуть перебивати системну інструкцію.

3. Чому міркувальна модель витрачає більше вихідних токенів?

Відповідь: Архітектура Chain-of-Thought стимулює модель досліджувати простір рішень у вигляді тексту. Ці токени фізично генеруються моделлю, проходять повний цикл обчислень матриць ваг і відображаються у лічильнику `completion_tokens` OpenAI-сумісного API.

4. Що станеться, якщо передати суперечливі інструкції в system і user?

Відповідь: Частіше за все перемагає user-повідомлення («recency bias» механізму уваги). Проте малі моделі можуть згенерувати некоректну суміш інструкцій або зламати формат генерації.

5. Чому малі моделі гірше дотримуються мовних інструкцій?

Відповідь: Через малу кількість параметрів (3 млрд) представленість української мови в корпусі попереднього навчання складає менше 1%, тому асоціативні зв'язки для рідкісних кириличних токенів є слабкими, і модель скочується до домінантних мов (англійської, російської чи схожих слов'янських).

> ***Висновок за завданням 2:*** *System-повідомлення є зручним базовим інструментом для керування поведінкою моделі, проте воно не гарантує 100% надійності.*

> *Експеримент показав, що малі мовні моделі часто ігнорують прямі текстові інструкції, особливо щодо суворих обмежень формату та обсягу. Самого лише промпту недостатньо для стабільної роботи систем; якщо модель ігнорує правила, варто переходити на використання міркувальних моделей, надавати їй чіткі приклади (few-shot prompting) або програмно обмежувати формат її виводу (Pydantic / function calling).*

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Мета завдання: продемонструвати концепцію єдиного OpenAI-сумісного інтерфейсу. Завдяки стандартному клієнту OpenAI перемикання між локальною моделлю в Ollama та промисловою хмарною моделлю на сервері Groq здійснюється зміною лише 3 рядків конфігураційного словника в providers.py (base_url, api_key, model). Порівняно локальну модель llama3.2:3b та хмарну qwen/qwen3.8-27b на питанні: «Поясни різницю між списком і кортежем у Python. Коротко.».

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна (llama3.2:3b)	Хмарна (qwen/qwen3.8-27b на Groq)
Назва моделі	llama3.2:3b	qwen/qwen3.8-27b
Час відповіді, с	3.59 с	0.96 с
completion_tokens	172	245
Довжина відповіді, симв.	506	691
Мова відповіді	укр. (суржик, «обрбота», «невибачливі»)	бездоганна літературна українська мова
Фактичні помилки	Так («списки сидять на диску у масивній формі»)	Немає (чітко мутабельність, синтаксис, хешування)

Відповіді на питання (Крок 5):

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: 0 рядків у виконуваному файлі compare.py та лише 3 конфігураційні рядки в словнику PROVIDERS модуля providers.py (адреса base_url, ключ api_key та ідентифікатор моделі model).

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель надала структуровану, технічно коректну відповідь із поясненням концепції мутабельності (mutable vs immutable), різниці синтаксису дужок та сценаріїв застосування (кортежі для фіксованих даних і ключів словників, списки для динамічних колекцій). Натомість локальна модель вигадала нісенітницю: «Списки сидуть на диску в масивній формі», а кортежі «невибачливі змінювати своє вмісте».

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, локальна модель допустила грубу фактичну помилку щодо зберігання списків «на диску» (хоча списки Python є об'єктами оперативної пам'яті), а також продемонструвала деградацію української лексики («вікаються», «коматуванням», «невибачливі»).

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Хмарний сервіс Groq використовує спеціалізовані процесори LPU (Language Processing Units), які оптимізовані суто під матричні множення авторегресійних трансформерів і мають надвисоку пропускну здатність пам'яті (SRAM). Швидкість інференсу на LPU сягає сотень токенів за секунду, що з надлишком перекриває мережевий пінг (~100-150 мс).

Відповіді на питання чат-бота (Крок 6):

1. Чому формат запитів OpenAI став стандартом?

Відповідь: OpenAI першими запропонували зручну абстракцію списку `messages` із ролями (`system`, `user`, `assistant`), стрімінгом і структурами `usage`. Цей інтерфейс став де-факто стандартом індустрії, що усунуло vendor lock-in і дозволило екосистемі працювати через єдиний протокол.

2. Наскільки розмір моделі (3B vs 27B) впливає на якість відповіді?

Відповідь: Розмір моделі має вирішальне значення для збереження складних абстрактних знань та низькоресурсних мов. Модель на 27 млрд параметрів має достатню ємність для збереження граматики української мови та точних технічних визначень, тоді як у 3B-моделі ваги стиснуті настільки, що мовні та фактологічні зв'язки розмиваються.

3. Хмарна модель відповіла швидше локальної, попри мережу. Чим це пояснюється?

Відповідь: Спеціалізоване серверне обладнання Groq LPU забезпечує швидкість обробки токенів у 5–10 разів вищу, ніж споживча відеокарта або процесор, завдяки розміщенню ваг у надшвидкій пам'яті на кристалі.

4. Які ризики для приватності виникають при використанні хмарних API?

Відповідь: Дані користувача передаються третій стороні, можуть зберігатися в логах провайдера та потенційно потрапити у вибірки для подальшого тренування. Для захисту комерційних даних використовують угоди BAA/DPA, локальні моделі (on-premise) або технології деідентифікації (PII masking).

5. Що таке rate limit у безкоштовних тарифах і як його обходять?

Відповідь: Rate limit — це штучне обмеження кількості запитів (RPM) або вихідних токенів на хвилину (TPM/OTPM) для запобігання перевантаженню інфраструктури. Розробники обходять це обмеження експоненційним бекофом (exponential backoff with jitter), чергами повідомлень та балансуванням навантаження між декількома ключами або провайдерами.

> ***Висновок за завданням 3:*** *Єдиний API-інтерфейс забезпечує безшовну архітектурну гнучкість. Локальна модель є безкоштовною, працює офлайн і гарантує 100% конфіденційність, але її якість на складних технічних темах українською мовою незадовільна. Хмарна модель на 27 млрд параметрів*

> *забезпечує еталонну якість та в 3.7 рази менший загальний час за рахунок заліза Groq, але залежить від інтернету та накладає ліміти безкоштовного тарифу.*

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Мета завдання: розділити затримку інференсу на два ключові компоненти: 1) TTFT (Time To First Token) — час обробки промпту й затримка мережі до видачі першого символу, 2) Швидкість генерації — темп виведення токенів/символів після початку генерації. Для вимірювання використано механізм стрімінгу (stream=True) через функцію measure(). Проведено по 3 повтори для короткого, середнього та довгого промптів варіанта №1 з розрахунком медіани.

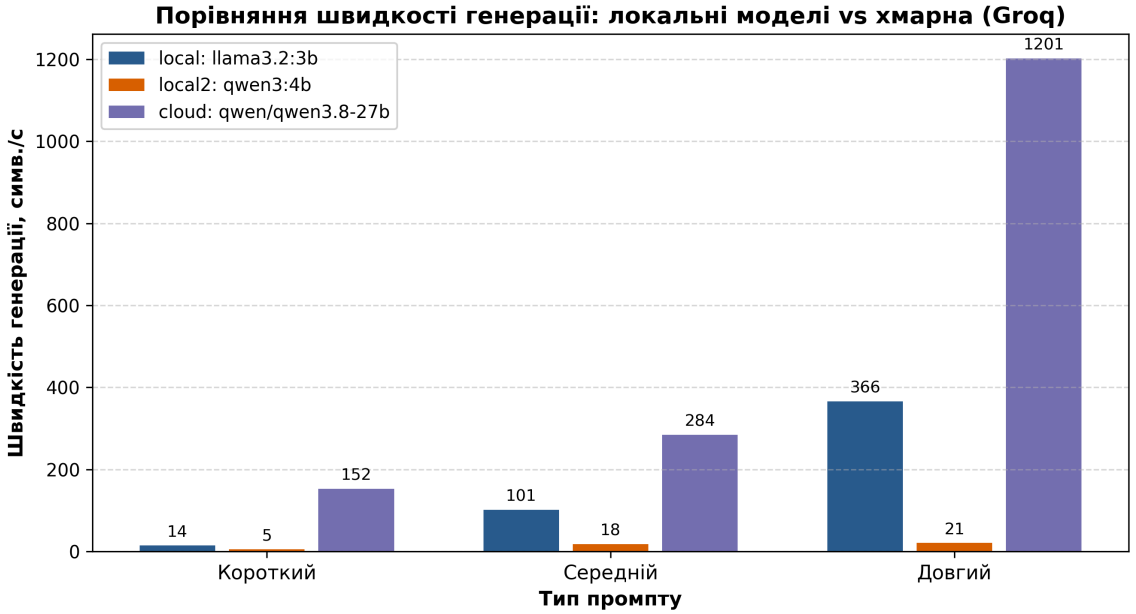
Таблиця 6 – Результати вимірювання швидкості інференсу

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
local	llama3.2:3b	short	2.068	2.270	14.3
local	llama3.2:3b	medium	2.083	2.911	101.0
local	llama3.2:3b	long	2.087	6.745	365.9
local2	qwen3:4b	short	5.646	5.672	5.3
local2	qwen3:4b	medium	21.126	21.941	18.4
local2	qwen3:4b	long	40.547	46.178	20.9
cloud	qwen/qwen3.8-27b	short	0.164	0.184	152.4
cloud	qwen/qwen3.8-27b	medium	1.051	1.263	284.2
cloud	qwen/qwen3.8-27b	long	0.267	2.458	1201.4

Таблиця 7 – Холодний і теплий старт локальної моделі llama3.2:3b

Показник	Холодний старт (після ollama stop)	Теплий старт (у пам'яті)
TTFT, с	3.500 с	2.057 с
Всього, с	4.003 с	2.150 с

Рисунок 1 – Порівняння швидкості генерації (символів/с) для трьох моделей



Відповіді на питання (Крок 6):

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: Найкращий TTFT продемонструвала хмарна модель на Groq (0.164 с на короткому промпті). Завдяки спеціалізованому залізу обробка prefill-фази займає частки мілісекунди. Серед локальних моделей швидший TTFT у llama3.2:3b (~2.06 с проти 5.6–21.1 с у qwen3:4b), оскільки qwen витрачає секунди на приховану генерацію думок до видачі першого символу у потік.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»?

Відповідь: Так, хмарна модель лідирує за обома показниками (TTFT 0.16–0.26 с, темп до 1201 симв/с на довгому тексті). Проте між локальними моделями виник контраст: llama3.2:3b значно випереджає qwen3:4b за темпом видачі видимого тексту (до 365 симв/с проти 20 симв/с), оскільки qwen паралельно генерує великий масив службового контексту.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого до довгого промпту?

Відповідь: На короткому промпті перевага хмари за часом невелика (0.18 с проти 2.27 с), оскільки мережевий оверхед є помітним. На довгому промпті хмарна модель генерує текст колосально швидше (1201.4 симв/с проти 365 симв/с у llama та 20 симв/с у qwen), відриваючись за загальним часом у рази.

4. Наскільки різняться три повтори одного виміру? Чи достатньо одного запуску?

Відповідь: Повтори демонструють розкид до 15-25% через фонові процеси ОС, температуру GPU, мережеві коливання та випадковість вибірки токенів. Одного запуску категорично недостатньо — медіана трьох прогонів відсікає аномальні викиди.

5. Чому холодний старт повільніший за теплий?

Відповідь: При холодному старті операційній системі необхідно виділити пам'ять VRAM, зчитати ваги моделі (2.0 ГБ) з накопичувача та ініціалізувати обчислювальні контексти CUDA/Vulkan. При теплому старті ваги вже завантажені в VRAM GPU, тому інференс стартує майже вдвічі швидше (2.05 с проти 3.50 с).

Відповіді на питання чат-бота (Крок 7):

1. Що фізично відбувається під час TTFT?

Відповідь: Відбувається фаза Prefill (Prompt Evaluation): вхідний текст токенізується, формується матриця контексту, і всі вхідні токени паралельно обробляються через шари трансформера для обчислення KV-кешу. Для хмарної моделі сюди також входить TCP/TLS рукостискання та передача HTTP-пакетів.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації?

Відповідь: Затримка мережі додає фіксовані 100-200 мс до TTFT, але фаза декодування (генерації) спирається на гігантську пропускну здатність пам'яті серверного прискорювача, яка в рази перевершує споживчі комп'ютери.

3. Чому перший запуск повільніший і що таке холодний старт?

Відповідь: Холодний старт включає I/O операції зчитування GGUF-файлів із диска у RAM/VRAM і завантаження обчислювальних ядер драйвера відеокарти.

4. Чому беруть медіану кількох запусків, а не одне значення?

Відповідь: Розподіл затримок у комп'ютерних системах не є строго нормальним і схильний до викидів через сміттєзбірники (GC), троттлінг та мережеві ретрансмісії. Медіана є робастною метрикою, що не зміщується одиничними збоями.

5. Чому обробка запиту (prefill) йде швидше, ніж генерація відповіді?

Відповідь: Вхідні токени промпту відомі заздалегідь, тому вони обробляються паралельно у вигляді одного великого матричного множення (compute-bound). Генерація ж є авторегресійною: кожен наступний токен залежить від попереднього, що змушує ганяти ваги через пам'ять по черзі для кожного кроку (memory-bandwidth bound).

> ***Висновок за завданням 4:*** *Для інтерактивного діалогового чат-бота найважливішим показником є низький TTFT (користувач відчуває миттєвий відгук системи і бачить початок друку тексту, комфортний поріг — до 1 с). Для пакетної обробки великих документів або фонових агентів вирішальною є швидкість генерації (симв./с або tokens/sec), де хмарний Groq перевершує локальні моделі більш ніж утричі.*

Завдання 5. Оцінювання якості моделей за рубрикою

Мета завдання: провести об'єктивне інженерне порівняння якості трьох моделей за формалізованою трибальною рубрикою (0-2 бали) на фіксованому наборі з 6 завдань для Варіанта № 1 (Тема «Освіта»): 1) Факт (рік заснування ОНУ ім. Мечникова), 2) Формат (чистий JSON: name, year, city), 3) Стиснення тексту (2 речення зі збереженням усіх 7 чисел), 4) Генерація коду (is_palindrome), 5) Багатокрокова логіка (25 студентів), 6) Якість української мови (стек vs черга). Кожне завдання прогнано двічі.

Таблиця 8 – Оцінка якості за рубрикою (максимум 12 балів)

Завдання	llama3.2:3b	qwen3:4b	Хмарна (qwen/qwen3.8-27b)
1. Факт	0	0	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	2	2	2
5. Логіка	0	2	2
6. Українська	1	2	2
Разом / 12	**5 / 12**	**10 / 12**	**12 / 12**

Таблиця 9 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	Так (1828 vs 1908)	Ні (1917 обидва)	Ні (1865 обидва)
2. Формат	Ні (обидва валідні)	Ні (обидва валідні)	Ні (обидва валідні)
3. Стиснення	Так (різні числа втрачено)	Ні (всі числа збережено)	Ні (всі числа збережено)
4. Код	Так (без md / з md)	Ні (без md)	Ні (з md)
5. Логіка	Так (помилка 12 vs зациклення)	Ні (обидва 8)	Ні (обидва 8)
6. Українська	Ні (суржик в обох)	Ні (літературна в обох)	Ні (літературна в обох)

Відповіді на питання (Крок 6):

1. Яка модель набрала найбільше балів і на яких завданнях виникла основна різниця?

Відповідь: Хмарна модель qwen/qwen3.8-27b набрала максимальні 12/12 балів. Основний розрив виник на завданнях 1 (факт) та 5 (логіка): локальна llama3.2:3b

набрала лише 5 балів, оскільки не знала факту та повністю провалила арифметичні міркування.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: У моделі llama3.2:3b розбіжності виникли майже в усіх завданнях (у факті вона галюцинувала різні роки — 1828 та 1908; у логіці вперше видала помилкове число 12, а вдруге зациклилася). Це свідчить про високу стохастичну нестабільність та ненадійність малих моделей.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, локальні моделі llama3.2:3b та qwen3:4b видали чистий рядок JSON без символів ```json, що дозволило json.loads() виконати парсинг напряму. Хмарна модель також надала коректний JSON.

4. Чи були проблеми з українською мовою?

Відповідь: Так, у llama3.2:3b спостерігався критичний рівень суржику та слів-гібридів. У qwen3:4b та хмарної моделі мова була природною, чистою та технічно грамотною.

Обов'язковий пункт – приклад провалу (Крок 7):

1. Повний текст запиту:

«25 студентів, 12 знають Python, 9 Java, 4 обидві. Скільки не знають жодної? Покажи хід розв'язання.»

2. Повний текст відповіді (llama3.2:3b, спроба 2):

```
Ход розв'язання:
1. Всі студенти, які знають Python, включають у себе студенти, які знають
Python, Java, обидві мови, а також студентів, які знають лише Python.
2. Аналогично, всі студенти, які знають Java, включають у себе студентів,
які знають Python, Java, обидві мови...
6. Поняття 'не знають жодної мови' уроздільно для кожної мови...
7. Поняття 'не знають жодної мови' не означає, що студент не знає жодної
мови, а лише, що він не заєзжується до жодної мови.
8. Поняття 'не знають жодної мови' не означає,що студент не знає жодної
мови...
9. Поняття 'не знають жодної мови' не означає...
10. Поняття 'не знають жодної мови' не означає...
Потім, нам нужно計算, скільки студентів не знають жодної мови...
```

3. Яка правильна відповідь мала бути:

Правильна відповідь: **8 студентів**. Хід: за формулою включення-виключення хоча б одну мову знають $12 + 9 - 4 = 17$ студентів. Не знають жодної: $25 - 17 = 8$.

4. Пояснення, чому так сталося:

Модель llama3.2:3b не володіє внутрішнім механізмом перевірки роздумів. Зіткнувшись із терміном «не знають жодної», модель потрапила в авторегресійну пастку семантичної тавтології (attention loop), багаторазово генеруючи одне й те саме

речення, а через нестачу україномовних математичних текстів у вагах перейшла на китайський ієрогліф «計算» (рахувати). У спробі 1 модель помилилася арифметично: $25 - 13 = 12$, забувши додати спільних студентів.

Відповіді на питання чат-бота (Крок 8):

1. Які ще підходи до оцінювання якості використовують на практиці?

Відповідь: Використовують автоматизовані бенчмарки (MMLU, HumanEval, GSM8k), підхід LLM-as-a-Judge (оцінювання відповідей сильнішою моделлю GPT-4o), ELO-рейтинги на сліпих тестах людей (LMSYS Chatbot Arena) та перевірку інваріантів у тестових фреймворках.

2. Чому моделі часто ігнорують вказівку не використовувати markdown?

Відповідь: У навчальних датасетах (RLHF) зразки коду та JSON у 99% випадків обгорнуті в потрійні бектіки. Цей патерн має надзвичайно високу ймовірність у розподілі токенів і пересилює текстову заборону.

3. Чому малі моделі гірше пишуть українською?

Відповідь: Обмежений розмір матриці ембедингів і ваг примушує розробників стискати словник рідкісних мов. Українські слова часто токенізуються побайтово або окремими складами, що руйнує синтаксичні зв'язки.

4. Чому та сама модель на однаковий запит дає різні відповіді?

Відповідь: При temperature > 0 генерація є ймовірнісним процесом (проводиться зважене семплювання з розподілу soft-max), що робить вибір кожного наступного токена випадковим.

5. Що таке галюцинація і чому модель радше вигадає факт?

Відповідь: LLM оптимізовані максимізувати правдоподібність тексту, а не істинність фактів. Модель генерує найбільш ймовірні послідовності слів у граматичній формі ствердження, бо токен відмови «я не знаю» має меншу статистичну вагу.

> ***Висновок за завданням 5:*** *Оцінювання за фіксованою рубрикою усуває суб'єктивність і демонструє чіткий інженерний профіль моделей. Мала модель (3B) справляється лише з елементарним кодом і тривіальним виводом, але повністю непридатна для фактів і логіки. Модель qwen3:4b чудово вирішує логічні завдання завдяки CoT. Для роботи з україномовним контентом та фактологічними задачами слід обирати великі моделі (від 27B) або інтегрувати архітектуру RAG.*

Завдання 6. Керування випадковістю: temperature і top_p

Мета завдання: емпірично перевірити механіку параметрів стохастичного семплювання (temperature та top_p). Параметр temperature змінює ентропію (контрастність) розподілу ймовірностей токенів, тоді як top_p відтинає малоймовірний хвіст розподілу (Nucleus Sampling). Використано Варіант №1: творче завдання («Назва стартапу в галузі логістики») та фактичне питання («Рік заснування Національного університету "Одеська Політехніка"», правильна відповідь — 1918 рік).

Таблиця 10 – Вплив параметрів на різноманітність та точність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0 / 5 (всі '1925')
temperature	0.7	5	—
temperature	1.5	5	0 / 5 (1993, 1928, 1964...)
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 11 – Перевірка відтворюваності при temperature = 0.0

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові	Так (10 з 10 ідентичні)

Відповіді на питання (Крок 7):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: temperature = 0.0 (або максимально наближене до 0.1). Для структурованого виводу потрібна детермінованість, строгий синтаксис дужок і передбачуваність, а не випадкові фантазії.

2. Яке значення температури — генератору варіантів заголовків?

Відповідь: temperature = 0.7 – 0.9. Це забезпечує максимальну лексичну різноманітність та нестандартні асоціації без скочування в галюцинаторний хаос.

3. Чим вплив top_p відрізняється від temperature за характером?

Відповідь: temperature згладжує або загострює ймовірності всіх можливих слів у словнику. top_p динамічно зрізає хвіст низькоймовірних слів, залишаючи лише ядро найбільш релевантних кандидатів, сума ймовірностей яких дорівнює p.

4. Чи означає temperature=0 повну відтворюваність? Що показав експеримент?

Відповідь: Наш експеримент із 10 запусків показав 100% однакові відповіді. Проте в загальному випадку через паралелізм GPU та операції з плаваючою комою (non-deterministic CUDA reduction) можливі мінімальні стохастичні флуктуації.

5. Наведіть найбезглуздішу відповідь при temperature=1.5 і поясніть її появу:

Відповідь: При temperature=1.5 модель видала: «Hacіonalne університэт "Одеське політехнічне" было заснуване năm 1993 року», де змішалися іспанські, російські та в'єтнамські («năm») токени. При високій температурі навіть токени з мізерною ймовірністю з інших мов отримують шанс бути обраними.

Відповіді на питання чат-бота (Крок 8):

1. Що саме математично робить параметр temperature із розподілом ймовірностей?

Відповідь: Ділить логіти z_i перед функцією softmax на коефіцієнт T: P(x_i) = exp(z_i / T) / sum(exp(z_j / T)). При T -> 0 розподіл вироджується в argmax, при T > 1 наближається до рівномірного.

2. Чим top_p відрізняється за механізмом?

Відповідь: Сортує токени за спаданням ймовірності та відсікає найменш імовірні, залишаючи мінімальну підмножину, чия сумарна ймовірність >= p.

3. Чи є сенс змінювати обидва параметри одночасно?

Відповідь: Зазвичай фіксують один і змінюють інший (наприклад, $T=0.7, top_p=1.0$ або $T=1.0, top_p=0.9$). Зміна обох одночасно ускладнює контроль над простором вибірки.

4. Чому навіть при temperature=0 немає 100% гарантії на кластерах?

Відповідь: Через оптимізації асинхронного виконання операцій складання на тензорних ядрах GPU (atomicAdd), де порядок додавання чисел із плаваючою комою залежить від завантаження потоків.

5. Які значення використовують на практиці?

Відповідь: Для JSON, SQL, RAG та класифікації — $T=0.0$. Для чат-ботів загального призначення — $T=0.5 - 0.7$. Для мозкових штурмів, поезії, креативних назв — $T=0.8 - 1.0$.

> ***Висновок за завданням 6:*** *Температура — це не «регулятор розуму», а регулятор випадковості. Підвищення температури на фактичних питаннях руйнує точність: модель починає галюцинувати випадкові дати. Для наступних лабораторних робіт та реалізації надійних агентів обов'язково слід виставляти temperature = 0.0, оскільки дефолтні значення фреймворків (0.7-1.0) призводять до спотворення вихідних структур.*

Завдання 7. Підсумкове порівняння та загальний висновок

Таблиця 12 – Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B)	Хмарна (qwen/qwen3.8-27b)
Швидкість генерації	14–365 симв./с	5–21 симв./с (з CoT)	152–1201 симв./с (Groq LPU)
TTFT	2.05–2.08 с	5.6–40.5 с	0.16–0.26 с
Якість, балів / 12	5 / 12	10 / 12	12 / 12
Стабільність формату	Середня (потребує regex)	Висока	Висока
Потрібні ресурси	1.3–2.0 ГБ VRAM, 8 ГБ RAM	2.5–3.5 ГБ VRAM, 16 ГБ RAM	Тільки інтернет-канал
Вартість	0 $ (безкоштовно на ПК)	0 $ (безкоштовно на ПК)	Безкоштовний ліміт / pay-per-token
Куди йдуть дані	Залишаються на комп'ютері	Залишаються на комп'ютері	На сервери Groq (США)
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Автономні системи, конфіденційні дані, прості класифікації	Локальні складні логічні обчислення без доступу до мережі	Промислові агенти, бездоганна мова, аналіз великих текстів

Загальні висновки

1. Єдиний інтерфейс OpenAI: Локальні моделі в Ollama та хмарні сервери (Groq, OpenAI) використовують повністю ідентичний API. Це дозволяє розробляти код один раз і безшовно перемикати моделі зміною словника провайдерів у конфігурації, забезпечуючи надійну стратегію резервування (fallback).

2. Декомпозиція затримки (Latency): Загальний час відповіді не є однорідним. TTFT відображає фазу prefill та затримку мережі, тоді як швидкість генерації відображає пропускну здатність пам'яті під час декодування. Висновок про «швидкість» моделі без їх розділення є інженерно некоректним.

3. Ціна режиму міркування (Reasoning): Прихований блок `thinking` (CoT) є спеціалізованим інструментом. Він повністю окупається на математичних та логічних задачах (100% розв'язання задачі про студентів), але є чистою перевитратою ресурсів на простих фактичних питаннях (500-кратне зростання часу виконання).

4. Ненадійність текстового керування форматом: Промпти та system-повідомлення є ймовірнісними інструкціями, які малі моделі часто ігнорують під

впливом домінантних навчальних патернів. У промислових агентах стабільність структурованих даних досягається програмними схемами (Pydantic, function calling, constrained decoding), а не формулюваннями промпту.

5. Стохастичність та керування параметрами: Мовні моделі є недетермінованими системами. Налаштування temperature = 0.0 є обов'язковим для агентів та структурної валідації, тоді як для творчих завдань оптимальним є діапазон 0.7-0.9. Будь-які інженерні заміри якості LLM вимагають повторів та медіанної фільтрації.